

==> Stored Procedures

1. Create a stored procedure that takes customer_id and returns a result set containing the full name, email, full address, and city of that customer.

```
DELIMITER /
CREATE PROCEDURE get_customer_data(IN id INT)
BEGIN
    SELECT CONCAT(cu.first_name, ' ', cu.last_name) AS full_name,
           cu.email,
           a.address AS full_address,
           ci.city
    FROM customer cu
    JOIN address a ON cu.address_id = a.address_id
    JOIN city ci ON a.city_id = ci.city_id
    WHERE cu.customer_id = id;
END /
DELIMITER;
```

2. Create a stored procedure that takes a start_date and end_date and returns the total revenue generated by each store during that period.

```
DELIMITER /
CREATE PROCEDURE total_revenue(IN start_date DATE, IN end_date DATE)
BEGIN
    SELECT store.store_id, SUM(p.amount)
    FROM payment p
    JOIN staff ON p.staff_id = staff.staff_id
    JOIN store ON staff.store_id = store.store_id
    WHERE p.payment_date BETWEEN @start_date AND @end_date
    GROUP BY store.store_id;
END /
DELIMITER ;
```

3. Create a stored procedure that takes a partial string (e.g., 'dark') and returns all films whose titles contain that string, along with their ratings.

```
DELIMITER /
CREATE PROCEDURE find_films_by_title(IN search_string VARCHAR(100))
BEGIN
    SELECT title, rating
    FROM film
    WHERE title LIKE CONCAT('%', search_string, '%');
END /
DELIMITER ;
```

4. Create a stored procedure that takes customer_id and returns the name of the category (e.g., 'Action') that the customer rents most often.

```
DELIMITER /
CREATE PROCEDURE get_customer_favorite_category(IN target_customer_id INT)
BEGIN
    SELECT c.name AS favorite_category
    FROM rental r
    JOIN inventory i ON r.inventory_id = i.inventory_id
    JOIN film_category fc ON i.film_id = fc.film_id
    JOIN category c ON fc.category_id = c.category_id
    WHERE r.customer_id = target_customer_id
    GROUP BY c.name
    ORDER BY COUNT(r.rental_id) DESC
    LIMIT 1;
END /
DELIMITER ;
```

==> Trigger

5. Create a BEFORE UPDATE trigger on address table to set last_update = NOW() whenever a row is changed.

```
DELIMITER /
CREATE TRIGGER before_address_update
BEFORE UPDATE ON address
FOR EACH ROW
BEGIN
    SET NEW.last_update = NOW();
END /
DELIMITER ;
```

6. Create a trigger to prevent an UPDATE to the staff table if the username is changed to 'admin'.

```
DELIMITER /
CREATE TRIGGER prevent_admin_username
BEFORE UPDATE ON staff
FOR EACH ROW
BEGIN
    IF NEW.username = 'admin' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: The username "admin" is reserved and cannot be
used.';
    END IF;
END /
DELIMITER ;
```

7. Create a BEFORE INSERT trigger on the customer table to ensure that any email address entered contains both an @ and a . symbol.

```
DELIMITER /
CREATE TRIGGER validate_customer_email
BEFORE INSERT ON customer
FOR EACH ROW
BEGIN
    IF NEW.email NOT LIKE '%@%.%' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Invalid email format. Must contain "@" and "."';
    END IF;
END /
DELIMITER ;
```

8. Create a table film_price_history. Create an AFTER UPDATE trigger on the film table that records the old price, new price, and change date whenever the rental_rate is modified.

```
CREATE TABLE film_price_history (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    film_id SMALLINT UNSIGNED,
    old_price DECIMAL(6,2),
    new_price DECIMAL(6,2),
    change_date DATETIME,
    FOREIGN KEY (film_id) REFERENCES film(film_id)
```

);

DELIMITER /

CREATE TRIGGER after_film_price_update

AFTER UPDATE ON film

FOR EACH ROW

BEGIN

IF OLD.rental_rate != NEW.rental_rate THEN

INSERT INTO film_price_history (film_id, old_price, new_price, change_date)

VALUES (OLD.film_id, OLD.rental_rate, NEW.rental_rate, NOW());

END IF;

END /

DELIMITER ;